AI AGENTS, MCP & APIS FOR PMS AND BAS · LESSON 8 OF 11

# OAuth 2.0 in Depth: Scopes, PKCE and Secure Tokens

Understand how OAuth 2.0 lets an app act for a user without seeing the user's password. Learn the authorisation code flow with PKCE, how scopes limit a token, how access and refresh tokens expire and renew, and how to store and protect tokens so that AI agents and integrations do not become a security hole.

Gap-fill lesson · ✦ 2026 · Advanced · ⏱ 120 min

**By the end of this lesson you can**

- Explain the roles in OAuth 2.0: resource owner, client, authorisation server and resource server
- Walk through the authorisation code flow with PKCE step by step
- Request the smallest set of scopes an integration needs
- Store, rotate and revoke tokens safely

## The problem OAuth solves

A bot that reads a user's email should not ask for the user's password. Passwords give full access, cannot be limited, and are hard to revoke without changing them everywhere. **OAuth 2.0** is the standard that lets a user approve a specific kind of access for a specific app, through the provider that holds the account. The app receives a **token**, which it presents instead of the password.

CASE STUDY

### A Slack bot that posts, but cannot read

A team wants a bot that posts daily summaries to one channel. The bot needs only the permission to post messages. With OAuth, the team approves a token limited to posting. If the bot is compromised, it cannot read private channels or change settings.

## The four roles

| Role | Who | In the Slack example |
|---|---|---|
| Resource owner | The person whose data is accessed | The workspace admin who approves |
| Client | The app that wants access | The summary bot |
| Authorisation server | Issues tokens after the user approves | Slack's sign-in service |
| Resource server | Holds the data and checks tokens | Slack's API |

## The authorisation code flow with PKCE

For apps that run on a user's device or in a browser, the standard flow is the **authorisation code flow with PKCE** (Proof Key for Code Exchange, pronounced "pixy"). PKCE stops an attacker who intercepts the authorisation code from using it.

```
1. Client creates a random secret (code_verifier) and its hash (code_challenge)
2. Client sends the user to the authorisation server with:
      client_id, redirect_uri, scope, state, code_challenge, code_challenge_method=S256
3. User signs in and approves the requested scopes
4. Authorisation server redirects back with a one-time code and the state value
5. Client checks state matches (protects against forged redirects)
6. Client sends the code + code_verifier to the token endpoint
7. Server checks the verifier matches the challenge and returns:
      access_token (short-lived), refresh_token (optional), expires_in, scope
8. Client calls the resource server with: Authorization: Bearer <access_token>
```

Two values protect the flow. `state` is a random value the client checks on return, so that a forged redirect is rejected. The `code_verifier` must match the `code_challenge` sent earlier, so a stolen code is useless on its own.

Here is the token exchange in Python, shown as code and **not run**, because it needs a live provider:

Python

```python
# Illustrative only: replace the URLs and values with your provider's documentation.
import base64, hashlib, secrets, urllib.parse
import requests

verifier = base64.urlsafe_b64encode(secrets.token_bytes(32)).rstrip(b"=").decode()
challenge =
base64.urlsafe_b64encode(hashlib.sha256(verifier.encode()).digest()).rstrip(b"=").decode()

auth_url = "https://provider.example/oauth/authorize?" + urllib.parse.urlencode({
    "client_id": "YOUR_CLIENT_ID", "redirect_uri": "https://app.example/callback",
    "response_type": "code", "scope": "chat:write",
    "state": secrets.token_urlsafe(16), "code_challenge": challenge,
"code_challenge_method": "S256"})

# ... user approves, your callback receives ?code=...&state=...
resp = requests.post("https://provider.example/oauth/token", data={
    "grant_type": "authorization_code", "code": "CODE_FROM_CALLBACK",
    "redirect_uri": "https://app.example/callback", "client_id": "YOUR_CLIENT_ID",
    "code_verifier": verifier}, timeout=30)
token = resp.json()["access_token"]
```

## Scopes: give the smallest permission that works

A **scope** names a permission, such as `chat:write` or `orders:read`. The token carries only the scopes the user approved. Request the minimum:

| Integration | Needed scopes | Not needed |
|---|---|---|
| Daily summary poster | `chat:write` | `channels:read`, `users:read`, admin scopes |
| Order status lookup | `orders:read` | `orders:write`, `payments:*` |
| Ticket triage bot | `tickets:read`, `tickets:comment` | `tickets:delete`, `admin` |

Ask for more scopes later, when a feature needs them, and explain why. A user who sees "read and write everything" will refuse; one who sees "post messages" will say yes.

::::warn Admin tokens for an AI client An AI client that holds an administrator's token can do anything the administrator can do, including by mistake or under prompt injection. This is the single most common mistake with agent integrations. Give the agent a token for a service account with only the scopes the task needs. ::::

## Access tokens, refresh tokens and expiry

- **Access tokens** are short-lived (minutes to an hour). If one leaks, its useful life is limited.
- **Refresh tokens** are longer-lived and exchanged for new access tokens without the user signing in again. They are the most sensitive token, because they keep access alive.
- **Revocation** lets a user or administrator cancel access. Build a way to revoke a token from your admin page, and test it.

Python

```python
# Renewing an access token with a refresh token (illustrative; not run)
def refresh_access_token(refresh_token: str) -> str:
    resp = requests.post("https://provider.example/oauth/token", data={
        "grant_type": "refresh_token", "refresh_token": refresh_token,
        "client_id": "YOUR_CLIENT_ID"}, timeout=30)
    resp.raise_for_status()
    return resp.json()["access_token"]
```

## Storing and protecting tokens

| Do | Do not |
| --- | --- |
| Store tokens in a secrets manager or an encrypted database column | Store tokens in source code, config files committed to git, or client-side JavaScript |
| Keep access tokens in server memory or a short-lived cache | Put tokens in URLs, query strings or browser storage |
| Redact tokens from logs | Log full request headers |
| Rotate client secrets and review scopes regularly | Reuse one token across many systems |

## A checklist for every OAuth integration

1. The smallest scope set is documented with the reason for each scope.
2. PKCE and `state` are used for any flow involving a browser.
3. Tokens are stored in a secrets manager, not in code or logs.
4. Access tokens expire; refresh tokens are protected and revocable.
5. The integration uses a service account, not a personal admin login.
6. A test confirms that a revoked token is refused.

INTERVIEW CORNER

**How would you let an AI agent read a customer's support tickets safely?**

Use OAuth with a read-only scope such as `tickets:read` and a service account limited to the customer's team. Keep the token in a secrets manager, use short-lived access tokens, log each tool call without logging the token, and add a human approval step before any write action such as closing a ticket.

::::key What to remember - OAuth lets an app act for a user without the user's password. - The authorisation code flow with PKCE and the `state` value protect the sign-in. - Request the smallest set of scopes, and never give an AI client admin rights. - Access tokens expire; refresh tokens are the most sensitive and must be revocable. - Store tokens in a secrets manager and keep them out of logs, URLs and code. ::::

## Quick quiz

5 questions · tap an answer to check it

**1. What does the `scope` in an OAuth request define?**

A The time the token lasts

B The server that issues the token

C The permissions the token grants

D The user's password

**Answer: C.** Scopes limit what the token can be used for.

**2.** What does PKCE protect against?

A Slow networks

B An attacker who intercepts the authorisation code and tries to use it

C Expired passwords

D Duplicate scopes

**Answer: B.** The code verifier must match the challenge, so an intercepted code is not enough.

**3.** Why is the `state` value checked when the user returns?

A To count logins

B To reject a forged or mismatched redirect

C To choose the scopes

D To store the token

**Answer: B.** State ties the redirect to the request the client started.

**4.** Which token is the most sensitive, because it keeps access alive?

A The access token

B The authorisation code

C The refresh token

D The client ID

**Answer: C.** Refresh tokens are long-lived and can mint new access tokens.

**5.** A chatbot needs only to post messages. Which scope set is correct?

A Full administrator access

B The single posting scope, such as `chat:write`

C Read and delete for all channels

D The user's password

**Answer: B.** The smallest scope that works is the right choice.

EXERCISE **Design a token policy**

For an integration that reads order status and creates support tickets, write a one-page token policy: which scopes you request and why, how long access tokens last, where refresh tokens are stored, how a leaked token is revoked, and which log lines are safe to keep.

**Show solution (worked answer)**

Hints: separate read and write scopes into different service accounts if possible. Name the revocation step (who clicks, what happens to running jobs). Confirm that no log line contains the `Authorization` header.

## Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.